

C Programming Language

Pointers

- * 'pointer' එකක් යනු variable එකකි.
- * එහි value එක ලෙස store කරන්නේ තවත් variable එකක "memory address" එකයි.
- * pointers වලට ගැලපෙන 96 ඉතිරි ප්‍රධාන operators 2 ක් ඇති කරයි

(i) Dereferencing operator (*)

- ⊛ * pointer variable එකක් declare කිරීමට සහ
එම ග්‍රෑෆ් memory address එකෙන් තවත් value
එක access කිරීමට යොදයි.

~~⊛ Address operator (&)~~

(ii) Address operator (&)

- ⊛ variable එකක memory address එක ලබා
ගැනීමට සහ pointer variable එකකට තවත්
variable එකක memory address එක ලබා දීමට
යොදයි.

E.x. int m = 80;
 int *ptr = &m;
 printf("Value of m is : %d\n", m);
 printf("Address of m is : %p\n", &m);
 printf("Address of m is : %p\n", ptr);

- ⊛ Note : Memory address එකක් output කර ගැනීමට "%p" format specifier එක යොදයි. මෙයින් hexadecimal value එකක් ලෙස memory address එක ලැබේ.
මේ වෙනුවට "%d" යොදන්නොත්; warning or undefined behaviour එකක් ඇති වේ.

③ Structure pointer;

- ★ structure type યોજનાનો point નો પ્રકાર pointers હોય.

④ Function pointer;

- ★ function યોજનાનો point નો પ્રકાર.

- ★ data થી point ની રીતે લેવામાં આવેલા code યોજનાનો point નો પ્રકાર.

⑤ Double pointers;

- ★ યોજનાનો pointer યોજના memory address યોજના store નો પ્રકાર pointer યોજના.

- ★ 'pointer to pointer' જેવું ગણવામાં આવે.

- ★ જે યોજનામાં પ્રકાર "multi-level-pointers" ગણવામાં આવે છે.
(***ptr3 , ****ptr4)

⑥ NULL pointer;

- ★ કોઈ memory location યોજનાનો point નો સંદર્ભ નથી હોતો.

- ★ NULL value યોજનાનો assign કરવામાં આવે છે.

⑦ void pointer;

- ★ 'Void' type યોજનાનો ગણવામાં આવે છે.

- ★ કોઈ type યોજનાનો variable યોજનાનો point નો પ્રકાર હોય છે, કોઈ type યોજનાનો નો type cast કરવામાં આવે છે.

⑧ Wild pointer;

- ★ કોઈ assign નો પ્રકાર નથી હોતો.

- ★ કોઈ errors પ્રકાર હોય છે.

⑨ Constant pointers;

- ★ pointer એજાઈ store કરે memory address એજ constant એજાઈ. એજાઈ define કરે એજ modify કરે શકાયે.
- ★ કહો એજ એજ memory location એજ point કરે.

⑩ Pointer to Constant;

- ★ Constant variable એજાઈ point કરે એજ pointers એ.
- ★ Constant variable value એજ change કરે શકાયે એજ, pointer value એજ change કરે શકાયે.

Void pointers / Generic Pointers

Ex. `int a = 10;`
 `int b = 'x';`
 `void *p = &a;` // void pointer holds address → int a
 `p = &b;` // void pointer holds address → char b

- ★ 'void pointers' dereference કરે શકાયે. એજાઈ; data type એજાઈ type cast કરે એજાઈ એજ dereference કરે શકાયે.

Structures

- ★ એજાઈએ એજાઈ primitive data types એજાઈએ એજાઈ એજાઈ એજાઈ user-defined data type એજાઈ.
- ★ structure એજાઈ એજાઈ "struct" keyword એજાઈ એજાઈ.
- ★ structure એજાઈ એજાઈ data items, 'members' એજાઈ એજાઈ.

Syntax

```
struct structure-name {
    data-type1 var-name1;
    data-type2 var-name2;
    :
    RICHARD
}
```

① Note : ທີ່ syntax ຂອງ "structure template" ມາດຕະຖານ,
"structure prototype" ຄື ມາດຕະຖານ.

* structures ມີໜ້າທີ່ ເພື່ອ ສ້າງ structure type ຂອງ ມັນ ຈາກ variables
ທີ່ມີຊື່ ໃດໜຶ່ງ. ທີ່ນີ້ ມັນ ມີ ຂັ້ນຕອນ ທີ່ ມັນ ມີ.

i.) structure template ຂອງ ມັນ ມີ:

```
struct structure_name {  
    data-type member_name1;  
    data-type member_name2;  
} variable1, variable2, ... ;
```

ii.) Structure template ຂອງ ມັນ ມີ variable declare ທີ່ນີ້:

```
struct structure_name variable1, variable2, ... ;
```

* structure members access ທີ່ນີ້ ມັນ ມີ dot operator(.) ທີ່ນີ້.

Syntax

```
structure_name.member1;  
structure_name.member2;
```

* structure members ມີ initial values ທີ່ນີ້ ມັນ ມີ declaration
step ທີ່ນີ້ ມັນ ມີ ຂັ້ນຕອນ.

* structure members initialize ທີ່ນີ້ ມັນ ມີ 3 ຂັ້ນຕອນ.

i.) using assignment operator.

```
E.x. struct myStructure member1; var1;  
var1.member1 = value1;  
var1.member2 = value2;
```

ii.) using initializer list.

```
E.x. struct structure_name str = {value1, value2};
```


iii) using designated initializer list.

```
E.x. struct structure_name str = {  
    .member1 = value1,  
    .member2 = value2,  
    .member3 = value3  
}
```

Structure pointer

* structure pointer ~~point~~ point ~~to~~ structure members access ~~using~~, "arrow operator ($\rightarrow$)" ~~use~~.

```
E.x. struct Point {  
    int x, y;  
};
```

```
int main() {  
    struct Point str = {1, 2};  
    struct *Point *ptr = &str;
```

```
    printf("%d %d", ptr->x, ptr->y);
```

```
    return 0;
```

```
}
```

File handling

* C language ~~is~~ ^{is} ~~used~~ ^{used} ~~to~~ ^{to} ~~create~~ ^{create} files ~~in~~ ⁱⁿ 02 ~~ways~~.

(i) Text files (E.x. file.txt)

(ii) Binary files (E.x. file.bin)

① Text files;

A Text file contains data in the form of ASCII characters.

★ કદાચ line એકાંતે ગુજારે એકાંતે new line (\n) character એકાંતે.

★ text editor એકાંતે ડાટાએકાંતે હોવા એકાંતે સંકેતિત એકાંતે.

★ ".txt" file extension એકાંતે હોવા એકાંતે.

★ source code store એકાંતે હોવા એકાંતે.

② Binary files;

★ binary form એકાંતે data store એકાંતે હોવા એકાંતે.

★ Binary file એકાંતે create એકાંતે ડાટાએકાંતે program એકાંતે એકાંતે એકાંતે એકાંતે; content એકાંતે read એકાંતે એકાંતે એકાંતે program એકાંતે એકાંતે એકાંતે.

★ એકાંતે એકાંતે એકાંતે એકાંતે એકાંતે, એકાંતે એકાંતે એકાંતે.

★ ".bin" file extension એકાંતે હોવા એકાંતે.

File operations

① Create a new file → `fopen()` with attributes "a", "a+", "w", "w+"

② Open an existing file → `fopen()`

③ Read from a file → `fscanf()` or `fgets()`

④ Write in a file → `fprintf()` or `fputs()`

⑤ Moving to a specific location in a file → `fseek()` or `rewind()`

⑥ Closing a file → `fclose()`

File pointer

★ file pointer variable એકાંતે એકાંતે "FILE" macro હોવા એકાંતે.

Ex. FILE *pointer-name;

Open a file

* `fopen()` function 2227 05020 6065.

Syntax

`FILE* fopen( file_path , mode );`

ⓐ Note: If file opened successfully, returns a file pointer to it.
If not, returns NULL.

File opening modes

Mode	Description
<code>r</code>	read a file
<code>rb</code>	read a file in binary mode.
<code>w</code>	write in text mode
<code>wb</code>	write in binary mode.
<code>a</code>	append mode
<code>ab</code>	append in binary mode.

Ex. `FILE *ptr;`

`ptr = fopen( "filename.txt", "r" );`

`if ( ptr == NULL ) {`

`printf("The file is not opened. The program will now
exit.");`

`exit(0);`

`}`